

LAPORAN Sistem Pararel dan terdistribusi



Pengembangan Sistem Sinkronisasi Terdistribusi

Disusun Oleh :

Muhammad Abdurrachim 11231048

Youtube <https://youtu.be/PyzNjkaRVaA>

**PROGRAM STUDI INFORMATIKA
JURUSAN TEKNIK ELEKTRO, INFORMATIKA, DAN BISNIS
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN
2026**

DAFTAR ISI

DAFTAR ISI.....	2
1. Pendahuluan.....	3
2. Arsitektur.....	3
3. Algoritma.....	4
A. Distributed Lock (Algoritma Raft).....	4
B. Distributed Queue (Consistent Hashing).....	4
C. Cache Coherence (Protokol MESI).....	5
4. Analisis Performa.....	5
5. Kesimpulan.....	5

1. Pendahuluan

Dalam pengembangan sistem terdistribusi, tantangan utama yang dihadapi adalah menjaga konsistensi data dan koordinasi antar node yang saling terhubung melalui jaringan. Sistem ini dikembangkan untuk mensimulasikan skenario dunia nyata di mana beberapa server (node) harus bekerja sama dalam mengelola sumber daya tanpa terjadi tumpang tindih atau kehilangan data.

Tujuan utama dari proyek ini adalah mengimplementasikan tiga pilar utama dalam sinkronisasi terdistribusi:

- **Distributed Lock Manager:** Menjamin akses eksklusif ke sumber daya bersama menggunakan konsensus.
- **Distributed Queue System:** Mengelola antrian pesan yang tersebar secara efisien dan tahan banting.
- **Distributed Cache Coherence:** Memastikan data pada memori cepat (cache) di setiap node tetap sinkron dan valid.

Sistem ini dirancang untuk memiliki sifat Fault Tolerant, di mana sistem tetap dapat beroperasi dengan benar meskipun salah satu node mengalami kegagalan.

2. Arsitektur

Sistem ini menggunakan arsitektur Multi-Node Cluster yang terdiri dari 3 node identik (node1, node2, node3) yang berjalan di dalam kontainer Docker terpisah.

Komunikasi Antar Node: Setiap node berkomunikasi menggunakan protokol HTTP/REST yang dibangun di atas pustaka aiohttp (Python). Komunikasi dilakukan secara asinkron (asyncio) untuk menangani banyak permintaan sekaligus dengan latensi rendah.

Discovery: Setiap node mengetahui alamat node lainnya melalui konfigurasi lingkungan (.env) yang mendefinisikan daftar CLUSTER_NODES.

State Management: Meskipun node bersifat otonom, status konsensus (Raft) disimpan di memori masing-masing node, sementara data antrian

(Queue) disimpan secara persisten di dalam Docker Volume agar tidak hilang saat kontainer dimulai ulang.

Orchestration: Docker Compose digunakan untuk menyatukan jaringan antar node dan memastikan layanan pendukung seperti Redis tersedia untuk keperluan koordinasi tambahan.

3. Algoritma

A. Distributed Lock (Algoritma Raft)

Untuk mengelola kunci (lock) secara terdistribusi, sistem menggunakan algoritma Raft Consensus. Prosesnya adalah sebagai berikut:

- **Leader Election:** Node-node akan melakukan voting untuk memilih satu Leader. Hanya Leader yang berhak menerima permintaan kunci (*acquire*) atau pelepasan kunci (*release*).
- **Log Replication:** Setiap kali ada permintaan kunci, Leader akan mencatatnya dalam log dan mengirimkan salinannya ke Follower.
- **Commitment:** Jika mayoritas node (kuorum) mengonfirmasi telah menerima catatan tersebut, kunci dianggap sah (*committed*) dan status kunci diperbarui di seluruh cluster.
- **Deadlock Detection:** Sistem memiliki logika tambahan untuk mendeteksi "Wait-For Graph". Jika terjadi siklus di mana dua node saling menunggu kunci satu sama lain, sistem akan mendeteksi dan melaporkannya agar tidak terjadi kemacetan permanen.

B. Distributed Queue (Consistent Hashing)

Untuk pembagian beban antrean, sistem menggunakan Consistent Hashing:

- **Hash Ring:** Semua node dipetakan ke dalam sebuah "lingkaran virtual" berdasarkan nilai hash unik mereka.
- **Ownership:** Setiap pesan/topik akan di-hash, dan posisinya di lingkaran akan menentukan node mana yang bertanggung jawab menyimpannya.
- **Scalability:** Jika node ditambah atau dikurangi, hanya sebagian kecil data yang perlu berpindah tempat, sehingga sistem sangat stabil.

- **Persistence:** Pesan yang masuk disimpan ke file JSON lokal agar tetap aman dari kegagalan sistem.

C. Cache Coherence (Protokol MESI)

Untuk menjaga sinkronisasi memori, sistem menggunakan protokol MESI:

- **Modified (M):** Node telah mengubah data secara lokal; node lain harus menganggap data mereka basi.
- **Exclusive (E):** Data hanya ada di satu node dan masih bersih (sama dengan sumber utama).
- **Shared (S):** Data ada di beberapa node dan semuanya masih valid untuk dibaca.
- **Invalid (I):** Data di node ini sudah basi karena node lain telah melakukan perubahan.

Sistem juga menerapkan kebijakan LRU (Least Recently Used) untuk menghapus data lama jika kapasitas cache sudah penuh.

4. Analisis Performa

Berdasarkan hasil benchmark, sistem menangani total 229 request dengan throughput rata-rata 5.97 request per detik pada konfigurasi 25 virtual user dan spawn rate 5 user per detik. Average latency keseluruhan berada pada 3654.43 ms, sedangkan p95 latency berada pada 20000 ms. Nilai p95 yang jauh lebih tinggi dari median menunjukkan bahwa sebagian request mengalami tail latency tinggi, terutama pada operasi yang membutuhkan komunikasi antar-node atau operasi yang menunggu respons lebih lama.

Dari total 229 request, terdapat 169 request berhasil dan 60 request gagal. Dengan demikian, success rate pengujian adalah 73.80%, sedangkan error rate adalah 26.20%. Seluruh request yang gagal berasal dari endpoint lock, yaitu 31 failure pada `POST /locks/acquire` dan 29 failure pada `POST /locks/release`. Endpoint queue dan cache tidak menghasilkan failure pada pengujian ini.

Distribusi request pada pengujian terdiri dari 139 request queue, 30 request cache, dan 60 request lock. Queue menjadi komponen dengan request terbanyak karena skenario Locust memberi bobot lebih besar

pada publish/consume. Cache memiliki jumlah request lebih sedikit, tetapi operasi write cache menunjukkan latency tertinggi dengan p95 28000 ms dan max latency 27705.93 ms. Lock memiliki failure tertinggi karena seluruh request acquire/release menghasilkan HTTP 409 Conflict.

Endpoint queue memiliki karakteristik performa yang dipengaruhi oleh consistent hashing. Pada topic `orders`, request diarahkan ke node owner tertentu. Hasil pengujian menunjukkan `POST /queue/orders/publish` memproses 50 request tanpa failure dengan rata-rata latency 5216.05 ms dan p95 20000 ms. Sementara itu, `POST /queue/orders/consume` memproses 45 request tanpa failure dengan rata-rata latency 1230.20 ms dan p95 1600 ms. Perbedaan ini menunjukkan bahwa operasi publish lebih berat dibanding consume pada pengujian ini, karena publish melibatkan pemilihan owner, forwarding bila diperlukan, dan persistence message ke storage lokal.

Operasi queue acknowledgement (`POST /queue/ack/{message\_id}`) juga stabil dengan 44 request, 0 failure, rata-rata latency 1086.00 ms, dan p95 1700 ms. Hal ini menunjukkan bahwa mekanisme ack berjalan baik pada pesan yang berhasil dikonsumsi. Karena skenario hanya memakai satu topic yaitu `orders`, distribusi beban belum sepenuhnya merata ke semua node. Jika jumlah topic diperbanyak, beban queue dapat tersebar lebih baik karena consistent hashing dapat memetakan topic berbeda ke node owner berbeda.

Operasi cache read memiliki latency lebih rendah dibanding cache write. `GET /cache/product-1` memproses 14 request tanpa failure dengan rata-rata latency 1243.66 ms dan p95 1500 ms. Sebaliknya, `POST /cache/product-1` memproses 16 request tanpa failure dengan rata-rata latency 14800.36 ms dan p95 28000 ms. Perbedaan ini terjadi karena write cache tidak hanya mengubah data lokal, tetapi juga melakukan propagasi invalidation dan update ke peer agar coherence antar-node tetap terjaga. Dengan demikian, cache read relatif ringan, sedangkan cache write memiliki overhead komunikasi yang lebih besar.

Operasi lock menunjukkan failure paling tinggi. `POST /locks/acquire` memiliki 31 request dan seluruhnya gagal, sedangkan `POST /locks/release` memiliki 29 request dan seluruhnya gagal. File `locust\_result\_failures.csv` menunjukkan error berupa HTTP 409 Conflict

pada kedua endpoint tersebut. Hal ini menandakan bahwa skenario Locust untuk lock belum ideal untuk mengukur performa lock normal, karena request lock membutuhkan kondisi leader Raft yang tepat dan state lock yang sesuai. Pada implementasi ini, operasi lock write harus dikirim ke leader. Jika request diarahkan ke node non-leader atau lock tidak berada pada state yang dapat dilepas, sistem mengembalikan konflik dan Locust menghitungnya sebagai failure.

Error rate keseluruhan sebesar 26.20% berasal dari 60 failure pada 229 request. Namun, failure tersebut terkonsentrasi pada endpoint lock. Endpoint queue dan cache pada pengujian ini tidak menghasilkan failure. Oleh karena itu, hasil benchmark menunjukkan bahwa queue dan cache stabil pada skenario yang diuji, sedangkan skenario lock perlu diuji ulang dengan mengarahkan request ke leader Raft dan mengatur owner/resource agar acquire dan release berjalan secara valid.

5. Kesimpulan

Dari hasil pengujian, sistem menunjukkan bahwa setiap komponen memiliki tradeoff performa yang berbeda. Queue dapat menjalankan publish, consume, dan ack tanpa failure pada skenario yang diuji, tetapi latency publish lebih tinggi karena melibatkan owner routing dan persistence. Cache read memiliki latency lebih rendah, sedangkan cache write memiliki latency tinggi karena membutuhkan propagation ke node lain agar coherence tetap terjaga. Lock manager belum berhasil diuji sebagai successful request pada skenario Locust ini karena seluruh request lock menghasilkan HTTP 409 Conflict.

Secara umum, queue dan cache dapat dikatakan stabil pada pengujian ini karena tidak menghasilkan failure. Namun, error rate keseluruhan masih tinggi karena endpoint lock gagal seluruhnya. Untuk mendapatkan hasil performance analysis yang lebih representatif, skenario Locust bagian lock perlu diperbaiki agar request dikirim ke leader Raft dan release hanya dilakukan setelah acquire berhasil.